

Computational Intelligence & Algorithm Optimization



1.1 Objetivos da Disciplina

Transformar vocês em solucionadores de problemas complexos, capazes de aplicar técnicas avançadas de inteligência computacional em cenários do mundo real.

O que vamos aprender?

Estudo e aplicação de técnicas de inteligência computacional e otimização aplicadas à resolução de problemas complexos.

Exploraremos:

- **Algoritmos Bioinspirados:** Técnicas que imitam processos da natureza, como evolução das espécies (Algoritmos Genéticos), comportamento de enxames (PSO) e colônias de formigas (ACO)
- **Metaheurísticas:** Estratégias de busca inteligente que, em vez de testar todas as possibilidades (o que seria inviável), navegam pelo espaço de soluções aprendendo com a experiência, equilibrando exploração de novas regiões e exploração de áreas promissoras. São como "atalhos inteligentes" para encontrar soluções de alta qualidade em problemas onde o número de combinações é astronômico.
- **Lógica Fuzzy:** Sistemas que lidam com incertezas e imprecisões, como o raciocínio humano
- **Otimização Multiobjetivo:** Como encontrar soluções que equilibram múltiplos objetivos conflitantes

Por que isso é importante?

Vivemos em um mundo onde problemas de otimização estão em todos os lugares:

- **Logística:** Como entregar 1.000 pacotes no menor tempo possível?
- **Finanças:** Como montar uma carteira de investimentos maximizando retorno e minimizando risco?
- **Indústria:** Como escalonar a produção de 500 peças em 10 máquinas?
- **Biologia:** Como entender o dobramento de proteínas?
- **Transporte:** Como reduzir o trânsito em uma cidade inteligente?

Para seu desenvolvimento profissional

Ao final da disciplina, vocês serão capazes de:

1. **Modelar problemas complexos** de forma matemática
2. **Selecionar a técnica adequada** para cada tipo de problema
3. **Implementar algoritmos** eficientes em Python
4. **Avaliar criticamente** os resultados obtidos
5. **Comunicar soluções** de forma clara para stakeholders

As empresas buscam profissionais que entendem não apenas como usar ferramentas, mas como criar soluções sob medida para problemas específicos. Este conhecimento é fundamental para carreiras em:

- Ciência de Dados e IA
- Pesquisa Operacional
- Engenharia de Software
- Consultoria Estratégica
- Empreendedorismo Tech

2. Avaliação Continuada (ACs), Valores e Uso do GitHub

Devem ser obrigatoriamente feitas em pares (se necessário trio para não ficar alguém sozinho(a)) e ocorre por meio de **Entregas Contínuas de Valor** (Avaliações Continuadas - ACs), permitindo o acúmulo gradual de nota por meio da prática semanal.

Tabela de Pontuação das ACs

Avaliação	Prazo / Data Marco	Tópicos Abrangidos	Pontuação Máxima
AC-1	26/08	Fundamentos de Otimização, Complexidade e Algoritmos Genéticos	Até 0,75 ponto

AC-2	16/09	Metaheurísticas Bioinspiradas (PSO e ACO)	Até 0,75 ponto
AC-3	21/10	Lógica Fuzzy e Sistemas de Inferência	Até 1,00 ponto
AC-4	18/11	Otimização Multiobjetivo (NSGA-II) e Consolidação	Até 1,00 ponto

ATENÇÃO: As entregas são em pares e as notas são individualizadas.

Formato dos Entregáveis

Todas as atividades práticas serão desenvolvidas em duplas ou trios no formato de Jupyter Notebooks (.ipynb), executados preferencialmente no Google Colab, VS Code local ou Code Space. Os arquivos devem ser nomeados no padrão: Aula_XX_datadodia.ipynb.

3. Metodologia 20/80 e Controle de Versão com GitHub

Como funcionam as aulas?

- 20% Teoria: Apresentação expositiva focada na intuição matemática, arquitetura de algoritmos e conceitos fundamentais.
- 80% Prática: Desenvolvimento ativo de código em laboratório, resolução de estudos de caso e implementação de desafios práticos.

Entregas Semanais via GitHub

Para simular o ambiente real de desenvolvimento de software e manter um portfólio profissional auditável, cada aluno deve criar o seu próprio repositório público no GitHub dedicado a esta disciplina.

Por que utilizar o GitHub individual?

1. Portfólio Profissional: O código desenvolvido semanalmente serve como comprovação pública de suas habilidades em Python, Otimização e Inteligência Computacional.
2. Rastreabilidade: Cada entrega é registrada com marcação de data e horário (*commits*), garantindo a autoria do trabalho.
3. Auditoria de Notas: As pontuações das ACs serão atribuídas com base nos arquivos postados em seu repositório oficial.

Após criar o repositório, envie o link do seu perfil/repositório através do formulário oficial disponibilizado pelo professor.

Nota: As provas individuais (AAS, Semestral, Exame) e o Projeto Tech possuem pontuações adicionais detalhadas na TABELA a seguir:

Avaliação	Data Prevista	Contexto
AC-1	26/08	Avaliação Continuada 1 - data final
AC-2	16/09	Avaliação Continuada 2 - data final
Prova Ind. (AAS)	23/09	Prova Individual
AC-3	21/10	Avaliação Continuada 3 - data final
AC-4	18/11	Avaliação Continuada 4 - data final
Prova Semestral	25/11	Avaliação Semestral
Projeto Tech	28/11	Entrega / Apresentação do Projeto
Prova Exame	09/12	Exame Final

4. Contextualização Prática do Exercício da Aula 01

Entender na prática o conceito de **Espaço de Busca** e a limitação da **Busca Aleatória (Monte Carlo)**.

O Problema: A Função Rastrigin

A Função Rastrigin é um benchmark clássico em otimização matemática. Ela possui um comportamento altamente acidentado, repleto de "mínimos locais" (vales onde algoritmos simples ficam presos) e apenas um único mínimo global absoluto na posição **(0,0)**, onde o valor da função é **0.0**.

Formulação bidimensional:

$$f(x,y) = 20 + (x^2 - 10 \cdot \cos(2\pi x)) + (y^2 - 10 \cdot \cos(2\pi y))$$

A nossa função **f de x e y** é composta por uma constante **20** somada a duas partes idênticas para cada variável:

- A parte em x é **x ao quadrado menos 10 vezes o cosseno de 2 pi x**;

- E a parte em y é y ao quadrado menos 10 vezes o cosseno de $2\pi y$."

Reparem na estrutura da Função Rastrigin: nós temos a constante **20** e depois aplicamos o mesmo termo quadrático com cosseno para a dimensão x — que é x^2 menos 10 cosseno de $2\pi x$ — e somamos com o exato mesmo termo para a dimensão y .

Nosso objetivo é testar uma estratégia de Busca Aleatória (gerar coordenadas x, y às cegas) para avaliar o quão eficiente (ou ineficiente) ela é para encontrar o ponto mínimo.

Para o código, usar o arquivo demonstração_aula01.py no repositório do professor.

Esse script serve para mostrar a diferença entre "chutar no escuro" e usar algoritmos de otimização inteligentes.

O que faremos no código:

1. **Mapear o Espaço de Busca:** Construimos a superfície 3D da Função Rastrigin no intervalo $[-5.12, 5.12]$ para visualizar os seus diversos vales (mínimos locais).
2. **Executar a Busca Aleatória (Random Search):** Sorteamos 1000 pares de coordenadas (x, y) aleatórias no terreno e calculamos a "altitude" (custo) de cada um.
3. **Guardar o Campeão:** A cada chute, testamos se o custo é o menor já visto (`if custo_atual < melhor_custo`). Se for, salvamos as coordenadas e o novo recorde.
4. **Mapear o Desempenho:** Cronometramos o tempo total de execução e geramos o gráfico 2D de convergência.

O que vamos validar no final:

- **Convergência Incompleta:** Como o mínimo global teórico é $(0, 0)$ com custo 0 , vamos validar que o chute aleatório chega perto, mas **difícilmente acerta o centro perfeito** em poucas tentativas.
- **Comportamento em Degraus:** No gráfico 2D, validamos que o algoritmo passa longos períodos estagnado sem encontrar melhorias.

- **Custo Computacional x Inteligência:** Validamos que a busca é extremamente rápida em tempo, mas **ineficiente em amostras** (desperdiça muitas das 1000 tentativas).